

코드로 배우는 리액트

6. 리액트와 상품 API 서버 연동

6장. 리액트와 상품 API 서버 연동

- 상품 API는 JSON 데이터 처리와 유사하나, 파일 데이터 추가로 처리 시간이 늘어남에 따라 모달 창 등의 추가 기능이 필요
- 컴포넌트 재사용을 통해 처리

개발목표

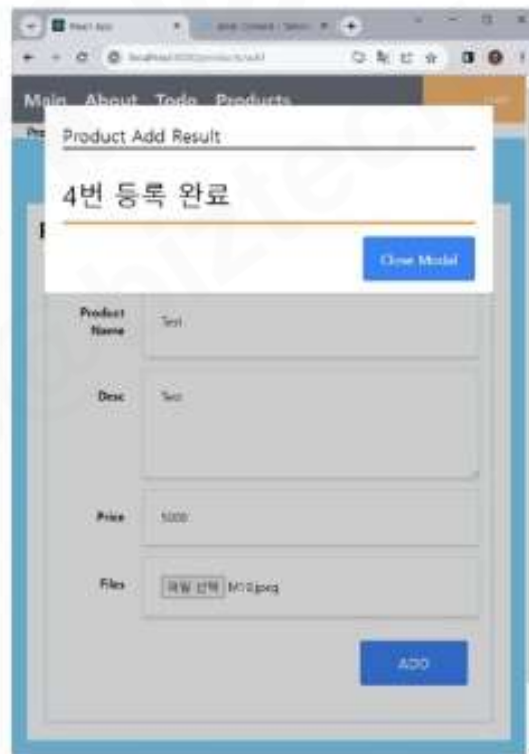
1. 파일이 추가되는 데이터의 처리
2. 기존 공통 컴포넌트들의 재사용

Index

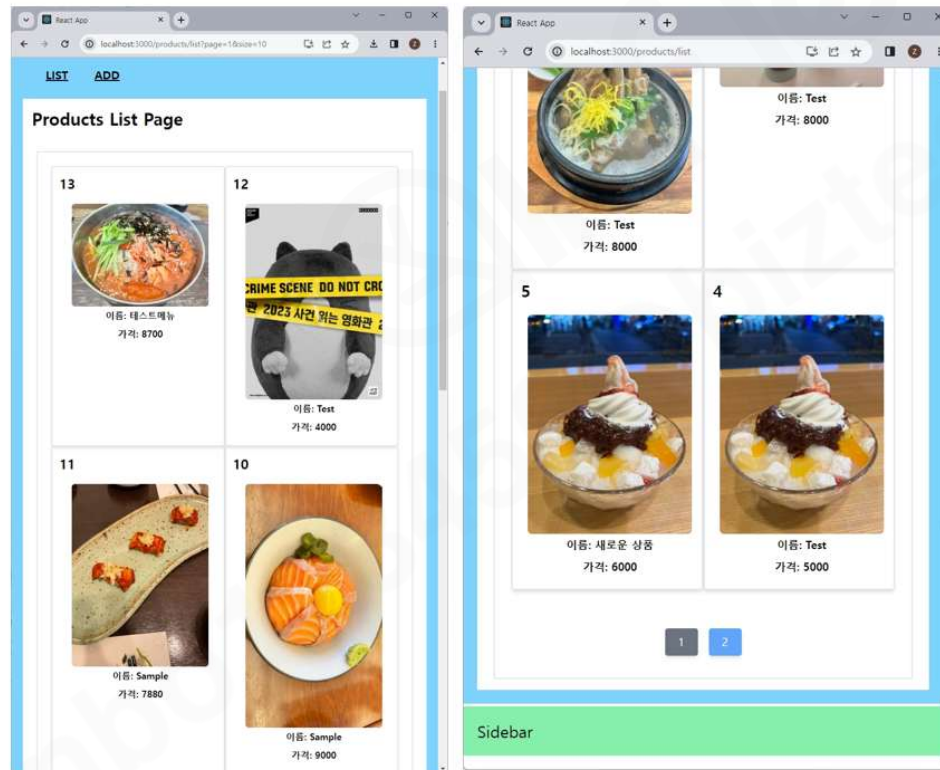
- 6.1 상품 관련 React-Router 설정
- 6.2 등록 페이지와 컴포넌트 처리
- 6.3 목록 페이지와 목록 컴포넌트 처리
- 6.4 조회 페이지와 조회 컴포넌트
- 6.5 수정/삭제 페이지와 컴포넌트 처리

useActionState 처리 방법

React Router의 Loader 설정



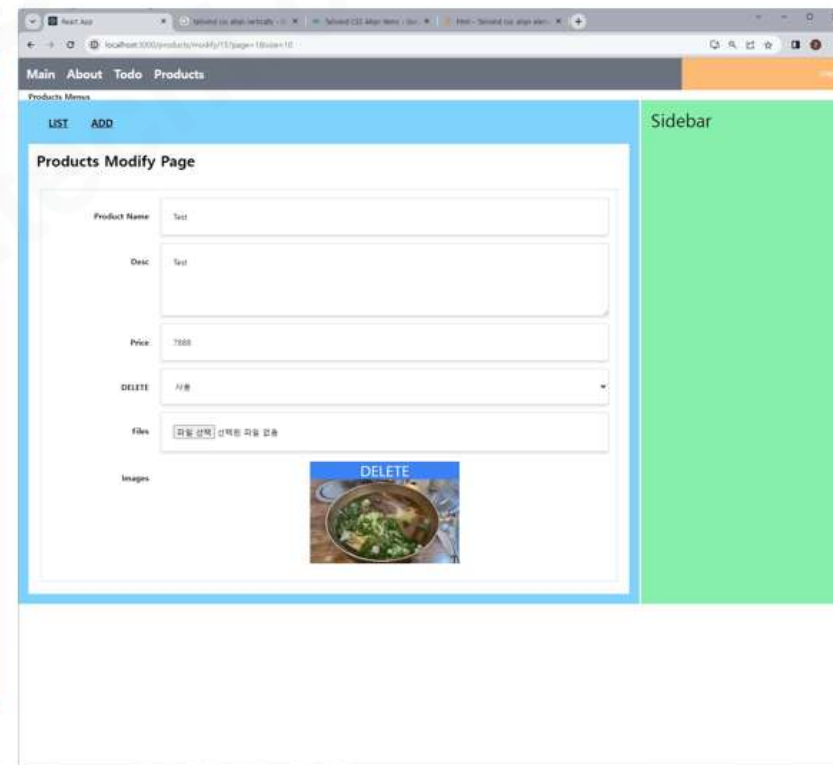
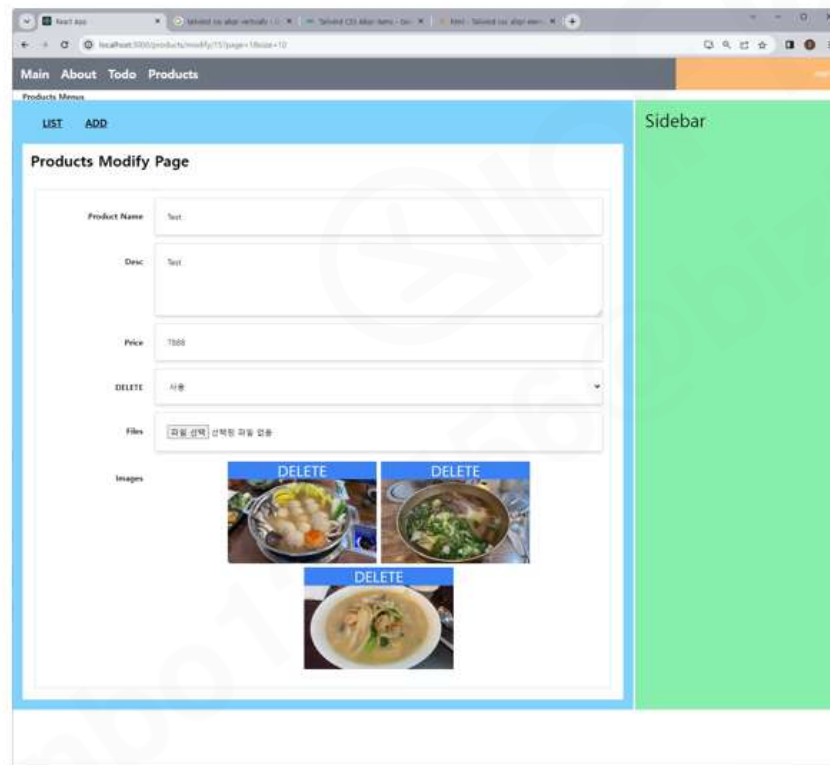
상품 관련 React-Router 설정



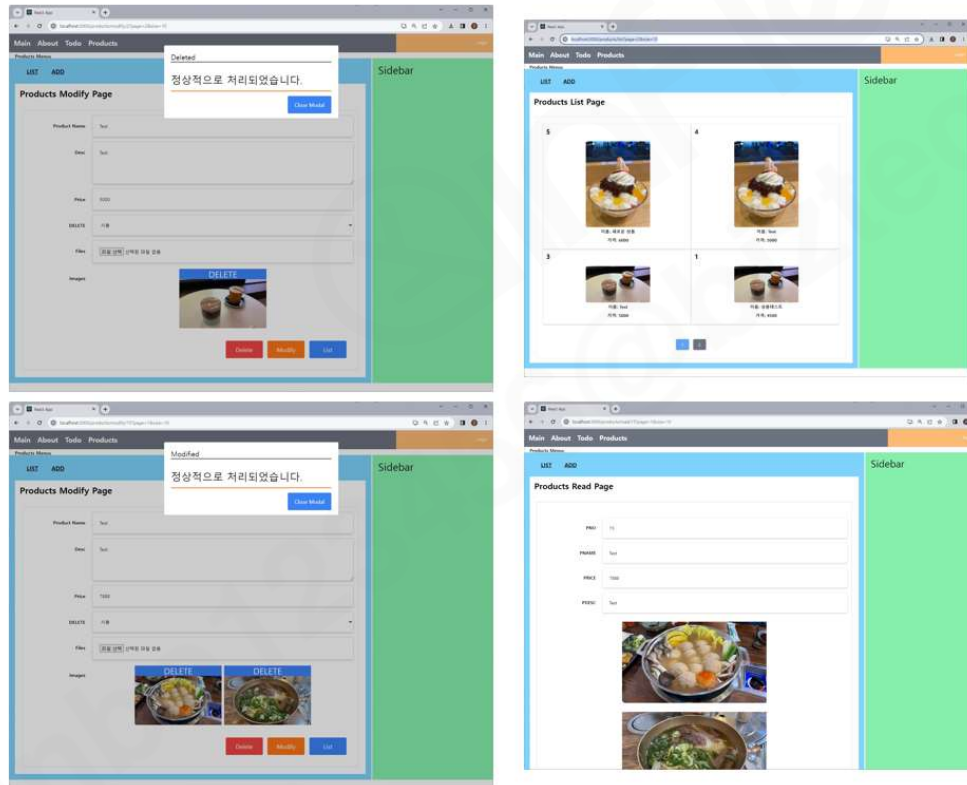
상품 관련 React-Router 설정



상품 관련 React-Router 설정

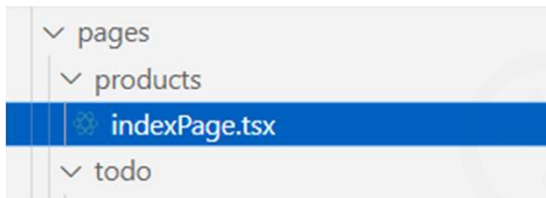


상품 관련 React-Router 설정



상품 관련 React-Router 설정

→ 상품 관련된 기능은 pages 폴더 내에 products 폴더를 생성



```
import { Outlet, useNavigate } from "react-router";

function IndexPage() {

  const navigate = useNavigate()

  const handleClickList = () => { navigate({ pathname: 'list' }) }
  const handleClickAdd = () => { navigate({ pathname: 'add' }) }

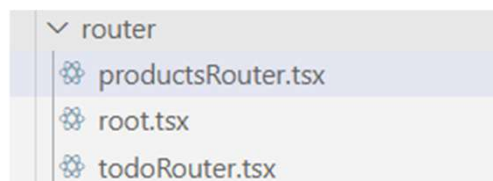
  return (
    <div>
      Products
      <div className="w-full flex m-2 p-2 ">
        <div
          className="text-xl m-1 p-2 w-20 font-extrabold text-center underline"
          onClick={handleClickList}
        >
          LIST
        </div>

        <div
          className="text-xl m-1 p-2 w-20 font-extrabold text-center underline"
          onClick={handleClickAdd}
        >
          ADD
        </div>
      </div>
      <div className="flex flex-wrap w-full">
        <Outlet/>
      </div>
    </div>
  );
}

export default IndexPage;
```

상품 관련 React-Router 설정

→ React-Router의 설정을 위해서 routes 폴더 내에 productsRouter.tsx 추가



```
import { lazy, Suspense } from "react"

const ProductsIndex = lazy(() => import("../pages/products/indexPage"))

const Loading = () => <div>Products Loading...</div>

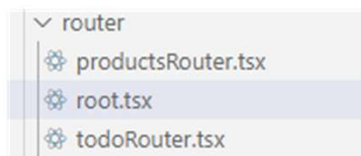
export default function productsRouter() {

  return (
    {
      path: "products",
      Component: ProductsIndex,
      children: [

    ]
  }
  )
}
```

상품 관련 React-Router 설정

→ root.tsx에 productsRouter.tsx 추가



```
import { Route } from "react-router";
import { lazy, Suspense } from "react";
import todoRouter from "../todoRouter";
import productsRouter from "../productsRouter";

const Loading = () => <div>Loading.....</div>

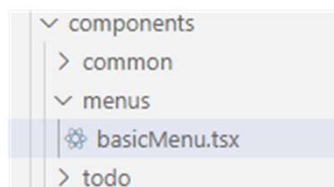
const Main = lazy(() => import("../pages/mainPage"))
const About = lazy(() => import("../pages/aboutPage"))

export default function rootRouter() {
  {
    path: "/",
    Component: BasicLayout,
    children: [
      { index: true,
        element: <Suspense fallback={<Loading/>}><Main/></Suspense>
      },
      { path: "about",
        element: <Suspense fallback={<Loading/>}><About/></Suspense>
      },

      todoRouter(),
      productsRouter()
    ],
  }
}
```

상품 메뉴의 추가

→ layouts/nav 폴더 내 BasicMenu.tsx 수정



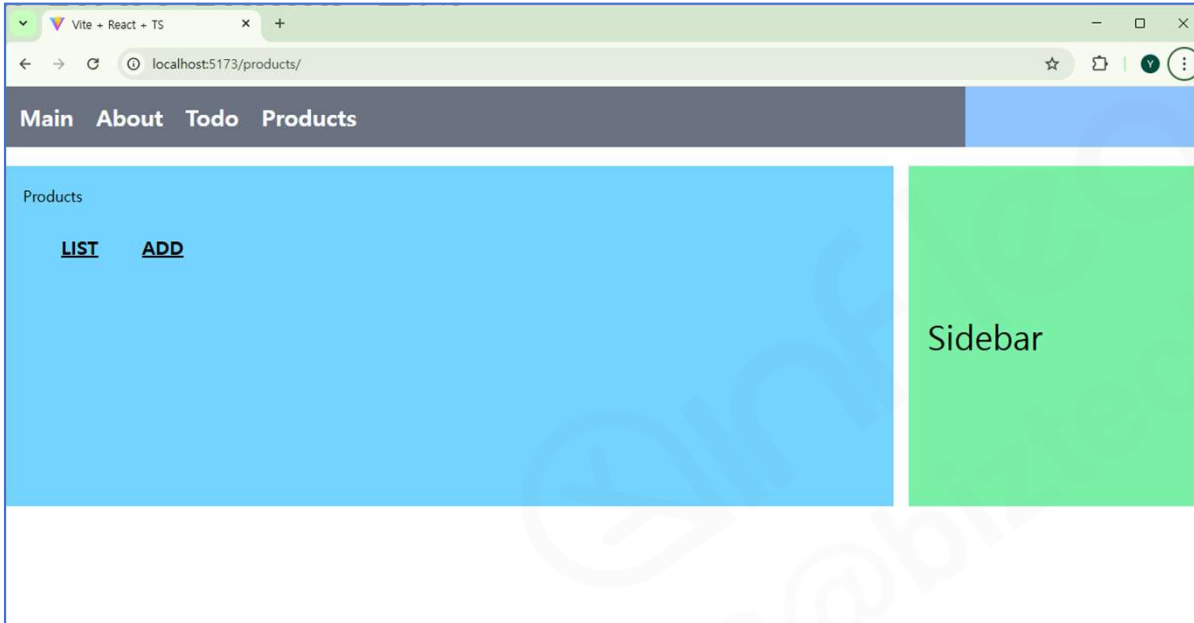
```
import { NavLink } from "react-router";

function BasicMenu() {
  return (
    <nav id='navbar' className="flex bg-blue-300">
      <div className="w-4/5 bg-gray-500" >

        <ul className="flex p-4 text-white font-bold">
          <li className="pr-6 text-2xl">
            <NavLink to="/">Main</NavLink>
          </li>
          <li className="pr-6 text-2xl">
            <NavLink to="/about">About</NavLink>
          </li>
          <li className="pr-6 text-2xl">
            <NavLink to="/todo/">Todo</NavLink>
          </li>
          <li className="pr-6 text-2xl">
            <NavLink to="/products/">Products</NavLink>
          </li>
        </ul>
      </div>
    </nav>
  )
}

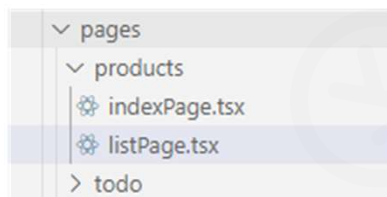
export default BasicMenu
```

01 상품 관련 React-Router 설정



ListPage

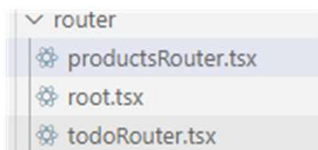
→ pages/products 폴더 내에 listPage.tsx를 추가



```
const ListPage = () => {  
  return (  
    <div className="w-full mt-4 border border-solid border-neutral-300 shadow-md">  
      <div className="text-2xl m-4 font-extrabold">  
        Products List Page  
      </div>  
    </div>  
  );  
}  
  
export default ListPage;
```

ListPage

→ productsRouter.tsx 에서 라우팅 설정



```
import { Navigate, Route } from "react-router";
import { lazy, Suspense } from "react";

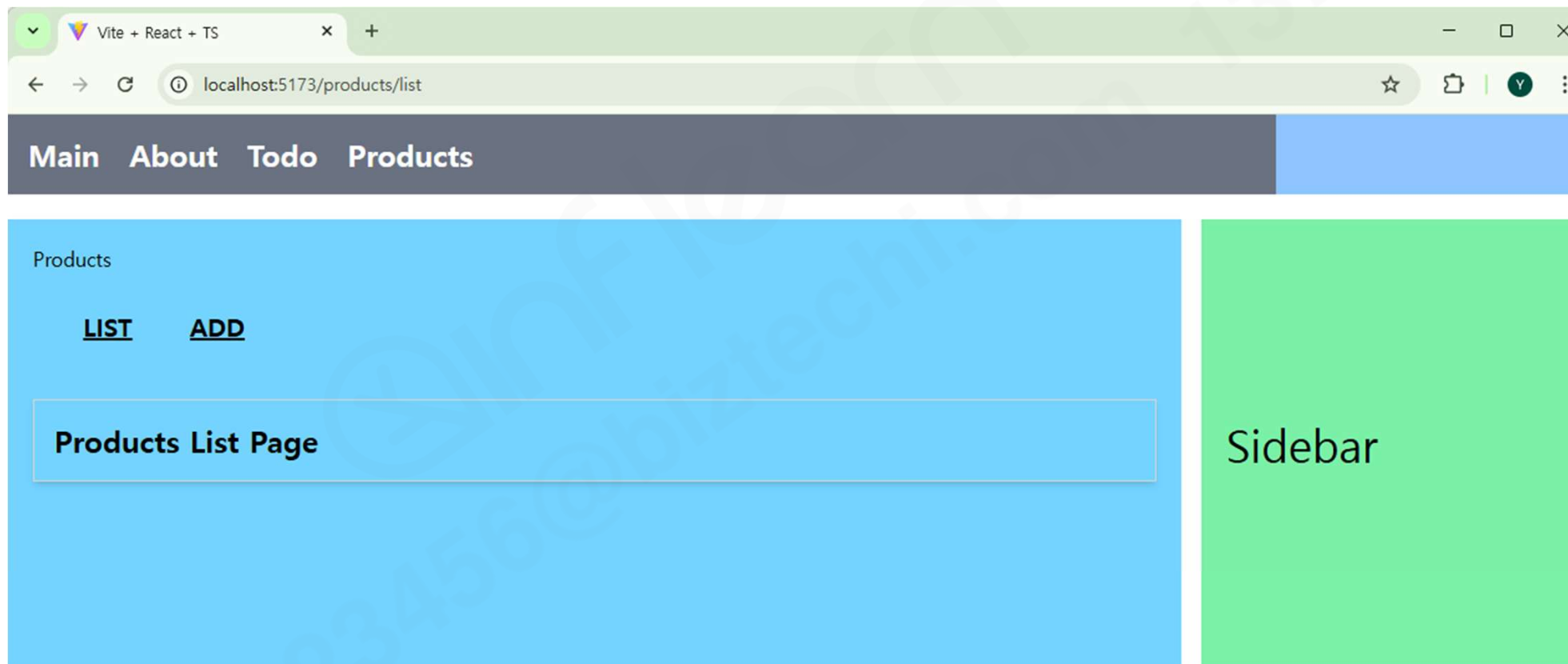
const Loading = () =><div>Loading...</div>

const ProductsIndex = lazy(() => import("../pages/products/indexPage"))
const ProductsList = lazy(() => import("../pages/products/listPage"))

const Loading = () => <div>Products Loading...</div>

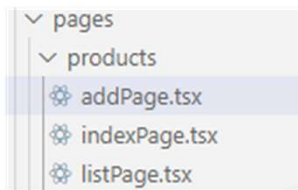
export default function productsRouter() {

  return (
    {
      path: "products",
      Component: ProductsIndex,
      children: [
        {
          path: "list",
          element: <Suspense fallback={<Loading/>}><ProductsList/></Suspense>
        }
      ]
    }
  )
}
```



등록 페이지와 컴포넌트 처리

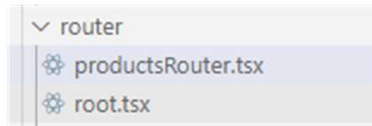
→ addPage.tsx



```
const AddPage = () => {  
  
  return (  
    <div className="p-4 w-full bg-white">  
      <div className="text-3xl font-extrabold">  
        Products Add Page  
      </div>  
    </div>  
  );  
}  
  
export default AddPage;
```

라우팅 설정

→ productsRouter.tsx



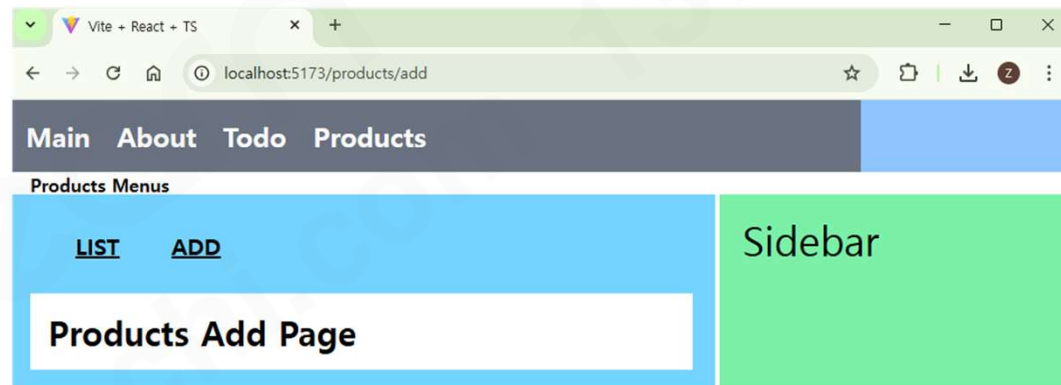
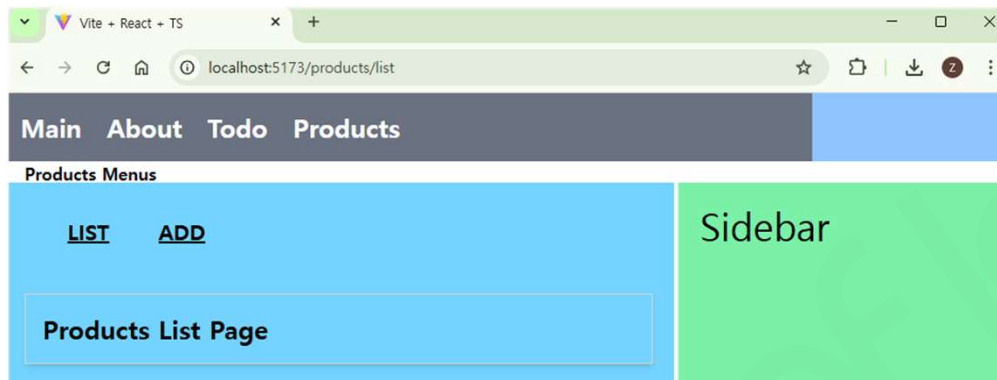
```
import { lazy, Suspense } from "react"
import { Navigate } from "react-router"

const ProductsIndex = lazy(() => import("../pages/products/indexPage"))
const ProductsList = lazy(() => import("../pages/products/listPage"))
const ProductsAdd = lazy(() => import("../pages/products/addPage"))

const Loading = () => <div>Products Loading...</div>

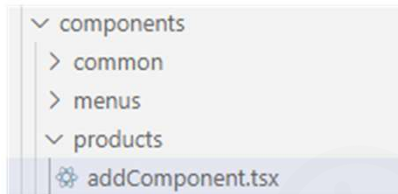
export default function productsRouter() {

  return (
    {
      path: "products",
      Component: ProductsIndex,
      children: [
        {
          path: "list",
          element: <Suspense fallback={<Loading/>}><ProductsList/></Suspense>
        },
        {
          path: "add",
          element: <Suspense fallback={<Loading/>}><ProductsAdd/></Suspense>
        },
        {
          path: "",
          element: <Navigate to={'/products/list'}></Navigate>
        }
      ]
    }
  )
}
```



상품의 AddComponent와 API 호출

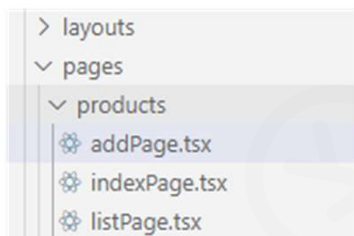
→ components 폴더에 products 폴더를 생성하고 AddComponent 생성



```
const AddComponent = () => {  
  
  return (  
    <div className = "border-2 border-sky-200 mt-10 m-2 p-4">  
      <div className="flex justify-center">  
        <h1>Add Component</h1>  
      </div>  
    </div>  
  );  
}  
  
export default AddComponent;
```

상품의 AddComponent와 API 호출

→ /pages/products/addPage.tsx 내 AddComponent를 추가



```
import AddComponent from "../../components/products/addComponent";

const AddPage = () => {

  return (
    <div className="p-4 w-full bg-white">
      <div className="text-3xl font-extrabold">
        Products Add Page
      </div>

      <AddComponent/>

    </div>
  );
}

export default AddPage;
```

useActionState()

- React 18.3이상
- 주로 <form> 처리를 위해 사용 - onChange - x
- 기본적으로 FormData를 이용할 수 있음

```
import axios from "axios"
import { useActionState } from "react"

interface ProductAddResult {
  result?: number,
  error? : string
}

const initState: ProductAddResult = {
  result: 0
}

const addAsyncAction = async ( state:ProductAddResult, formData:FormData): Promise<ProductAddResult> => {

  const pname = formData.get("pname") as string

  if(!pname) {
    return {error: "Insert Product Name"}
  }
  const res = await axios.post('http://localhost:8080/api/products/', formData )

  return {result: res.data.result}
}

const AddComponent = () => {

...이하
```

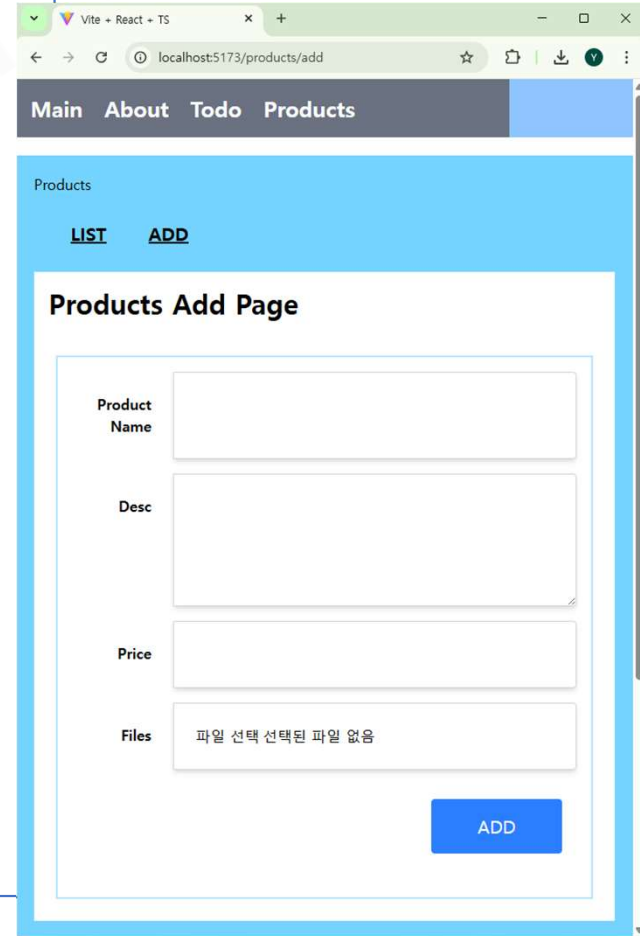
```
const AddComponent = () => {  
  
  const [state, action, isPending] = useActionState(addAsyncAction, initState )  
  
  return (  
    <div className = "border-2 border-sky-200 mt-10 m-2 p-4">  
  
      {isPending && <div>처리중.....</div>}  
  
      {state.result != 0 && <div>처리 결과 {state.result} </div>}  
  
      <form action={action} >  
        <div className="flex justify-center">  
          <div className="relative mb-4 flex w-full flex-wrap items-stretch">  
            <div className="w-1/5 p-6 text-right font-bold">Product Name</div>  
            <input className="w-4/5 p-6 rounded-r border border-solid border-neutral-300 shadow-md"  
              name="pname" required>  
            </input>  
          </div>  
        </div>  
        <div className="flex justify-center">  
          <div className="relative mb-4 flex w-full flex-wrap items-stretch">  
            <div className="w-1/5 p-6 text-right font-bold">Desc</div>  
            <textarea  
              className="w-4/5 p-6 rounded-r border border-solid border-neutral-300 shadow-md resize-y"  
              name="pdesc" rows={4} required>  
            </textarea>  
          </div>  
        </div>  
      </form>  
    </div>  
  )  
}
```

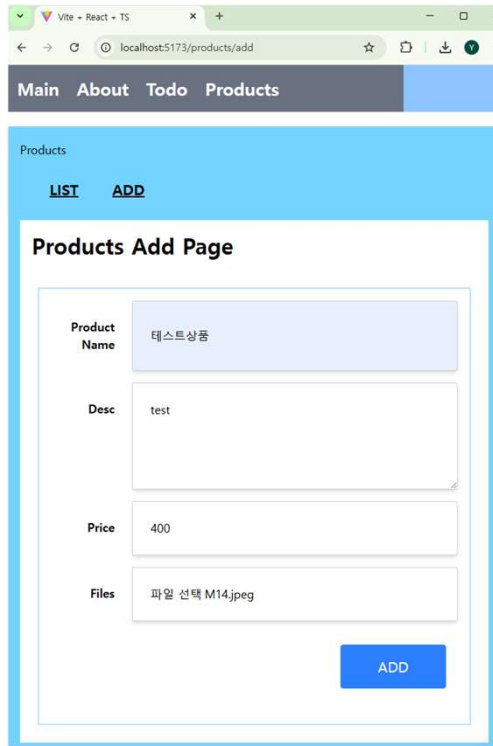
```

<div className="flex justify-center">
  <div className="relative mb-4 flex w-full flex-wrap items-stretch">
    <div className="w-1/5 p-6 text-right font-bold">Price</div>
    <input
      className="w-4/5 p-6 rounded-r border border-solid border-neutral-300 shadow-md"
      name="price" type={ 'number' } required>
    </input>
  </div>
</div>
<div className="flex justify-center">
  <div className="relative mb-4 flex w-full flex-wrap items-stretch">
    <div className="w-1/5 p-6 text-right font-bold">Files</div>
    <input
      className="w-4/5 p-6 rounded-r border border-solid border-neutral-300 shadow-md"
      type={ 'file' }
      name="files"
      multiple={ true }>
    </input>
  </div>
</div>
<div className="flex justify-end">
  <div className="relative mb-4 flex p-4 flex-wrap items-stretch">
    <button type="submit"
      className="rounded p-4 w-36 bg-blue-500 text-xl text-white">
      ADD
    </button>
  </div>
</div>
</form>
</div>
);
}

```

```
export default AddComponent;
```





Vite + React + TS

localhost:5173/products/add

Main About Todo Products

Products

LIST ADD

Products Add Page

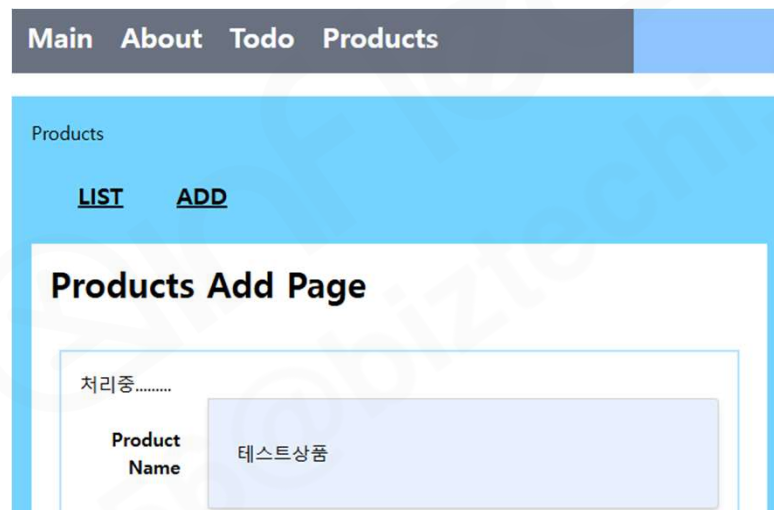
Product Name 테스트상품

Desc test

Price 400

Files 파일 선택 M14.jpeg

ADD



Main About Todo Products

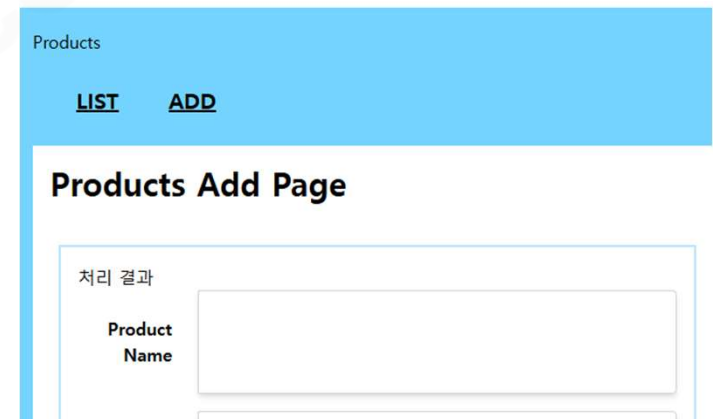
Products

LIST ADD

Products Add Page

처리중.....

Product Name 테스트상품



Products

LIST ADD

Products Add Page

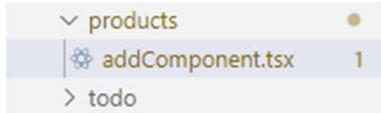
처리 결과

Product Name

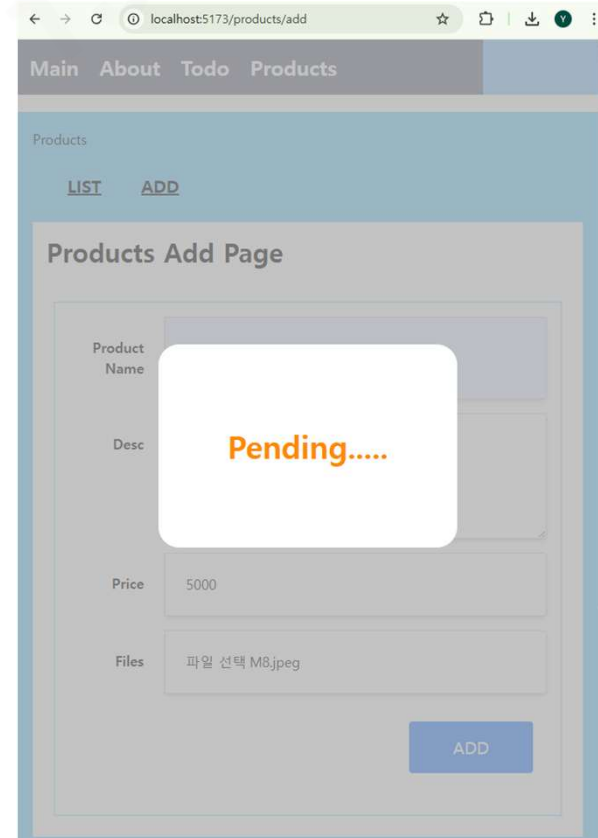
- 'Pending'을 표시하는 Pending 모달 창 제작하기



```
const PendingModal = ( ) => {  
  return (  
    <div  
      className={`fixed top-0 left-0 z-[1055] flex h-full w-full place-items-center justify-center bg-black`}   
      style={{ backgroundColor: 'rgba(169, 169, 169, 0.7)' }} >  
      <div  
        className="bg-white rounded-3xl opacity-100 min-w-min h-1/4 min-w-[600px] flex justify-center items-center ">  
  
        <div className="text-4xl font-extrabold text-orange-400 m-20">  
          Pending.....  
        </div>  
      </div>  
    </div>  
  );  
}  
  
export default PendingModal;
```

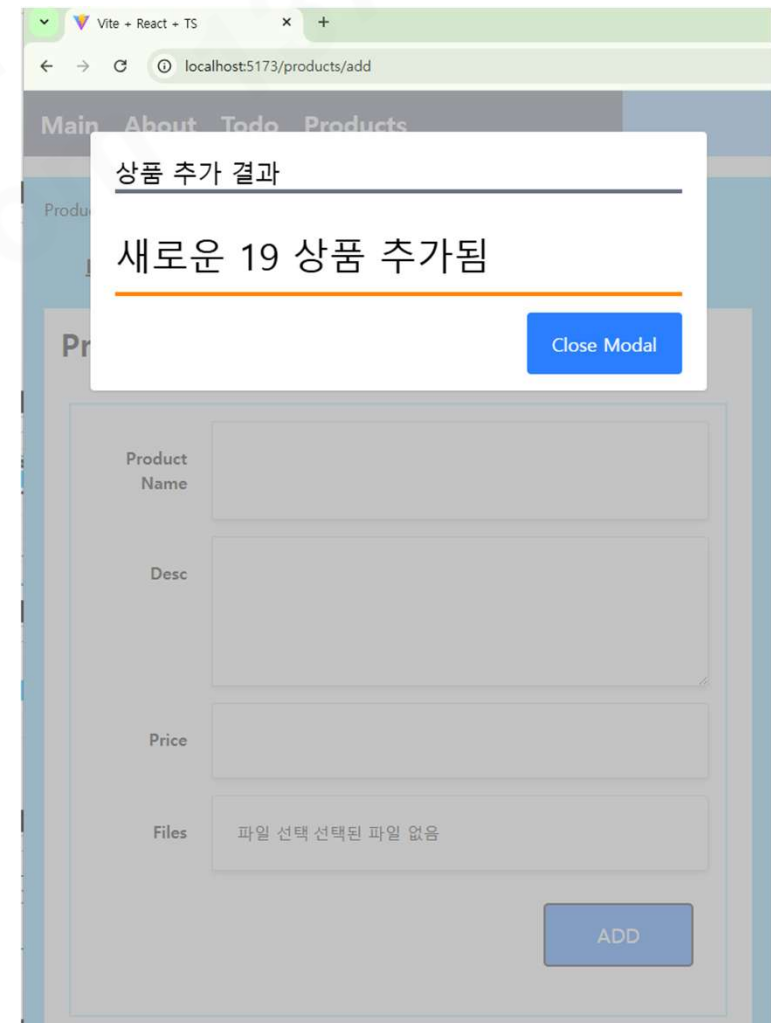


```
const AddComponent = () => {  
  const [state, action, isPending] = useActionState(addAsyncAction, initState )  
  
  return (  
    <div className = "border-2 border-sky-200 mt-10 m-2 p-4">  
      {isPending && <PendingModal/>}  
      {state.result !== 0 && <div>처리 결과 {state.result} </div>}  
    </div>  
  )  
}
```



- ResultModal의 재사용

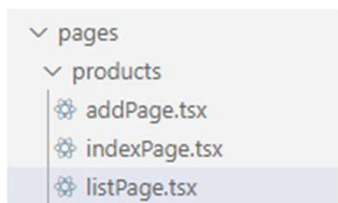
```
const AddComponent = () => {  
  const [state, action, isPending] = useActionState(addAsyncAction, initState )  
  const {moveToList} = useCustomMove()  
  const closeModal = () : void => {  
    moveToList()  
  }  
  
  return (  
    <div className = "border-2 border-sky-200 mt-10 m-2 p-4">  
      {isPending && <PendingModal/>}  
      {state.result !== 0 &&  
        <ResultModal  
          title="상품 추가 결과"  
          content={`새로운 ${state.result} 상품 추가됨`}   
          callbackFn={closeModal}/>  
      }  
    </div>  
  )  
}
```



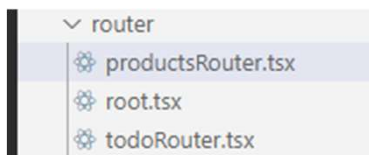
React Router 의 loader() 사용해 보기

- > 라우팅 시에 필요한 데이터를 로딩하는 함수 이용 가능
- > useNavigate()와 같이 이용하면 'Loading...' 처리 가능

- pages/products/listPage.tsx에 React Router가 사용할 loading 함수 추가



```
export async function loadProducts({request}: LoaderFunctionArgs) {  
  
  const url = new URL(request.url);  
  const page = url.searchParams.get('page') || "1";  
  const size = url.searchParams.get('size') || "10";  
  
  const queryStr = createSearchParams({page,size}).toString()  
  
  const res = await  
  axios.get(`http://localhost:8080/api/products/list?${queryStr}`)  
  
  return res.data  
  
}
```



```
import { lazy, Suspense } from "react"

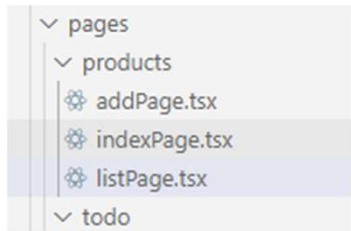
import { Navigate } from "react-router"

import { loadProducts } from "../pages/products/listPage"

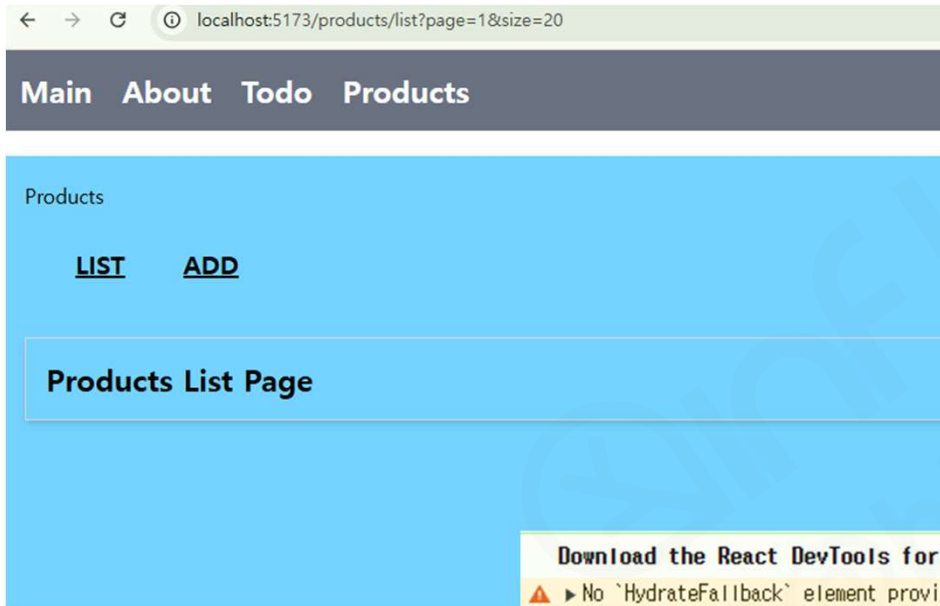
...
```

```
export default function productsRouter() {
  ...
  return (
    {
      path: "products",
      Component: ProductsIndex,
      children: [
        {
          path: "list",
          element: <Suspense fallback=<Loading/>><ProductsList/></Suspense>,
          loader: loadProducts
        },
        {
          path: "add",
          element: <Suspense fallback=<Loading/>><ProductsAdd/></Suspense>
        },
        {
          path: "",
          element: <Navigate to={'/products/list'}></Navigate>
        }
      ]
    }
  )
}
```

- listPage의 동작 확인 – useLoaderData()를 이용해서 처리된 결과 활용



```
const ListPage = () => {  
  
  const pageResponse = useLoaderData();  
  
  console.log(pageResponse)  
  
  return (  
    <div className="w-full mt-4 border border-solid border-neutral-300 shadow-md">  
      <div className="text-2xl m-4 font-extrabold">  
        Products List Page  
      </div>  
    </div>  
  );  
}  
  
export default ListPage;
```



Download the React DevTools for a better development experience: <https://react.dev/link/react-devtools>

▲ ▶ No `HydrateFallback` element provided to render during initial hydration

▼ {dtoList: Array(1), pageRequestDTO: {...}, totalCount: 10, pageNumList: Array(10), prev: false, ...} 1

router
productsRouter.tsx
root.tsx

```
return (  
  {  
    path: "products",  
    Component: ProductsIndex,  
    children: [  
      {  
        path: "list",  
        element: <Suspense fallback={Loading}><ProductsList/></Suspense>,  
        loader: loadProducts,  
        HydrateFallback: <div>로딩 중...</div>  
      },  
    ],  
  },  
)
```

- Product 에 맞는 타입 추가 및 출력 컴포넌트 listComponent작성

```
types
├── global.d.ts
├── products.d.ts
└── todo.d.ts
```

```
interface ProductDTO {
  pno: number;
  pname: string;
  price: number;
  pdesc: string;
  uploadFileNames: string[];
  delFlag: boolean;
}
```

```
products
├── addComponent.tsx
└── listComponent.tsx
```

```
import useCustomMove from "../../hooks/useCustomMove";
import PageComponent from "../../common/pageComponent";

function ListComponent({serverData}: {serverData:PageResponseDTO<ProductDTO>}) {

  const {moveToRead, moveToList} = useCustomMove()

  return (
    <div className="border-2 border-blue-100 mt-10 mr-2 ml-2 text-2xl">
      <div className="flex flex-wrap mx-auto p-6 bg-white">
        {serverData.dtoList.map(product =>
          <div
            key= {product.pno}
            className="w-1/2 p-1 rounded shadow-md border-2 border-gray-200"
            onClick={() => moveToRead(product.pno)}
          >
            <div className="flex flex-col h-full">
              <div className="font-extrabold text-2xl p-2 w-full ">
                {product.pno}
              </div>
              <div className="text-1xl m-1 p-2 w-full flex flex-col">
```

```

<div className="w-full overflow-hidden ">
  <img alt="product"
    className="m-auto rounded-md w-60"
    src={`http://localhost:8080/api/products/view/s_${product.uploadFileNames[0]}`}/>
</div>

<div className="bottom-0 font-extrabold bg-white">
  <div className="text-center p-1">
    이름: {product.pname}
  </div>
  <div className="text-center p-1">
    가격: {product.price}
  </div>
</div>
</div>
</div>
</div>
)}
</div>

<PageComponent serverData={serverData} movePage={moveToList}></PageComponent>

</div>
);
}

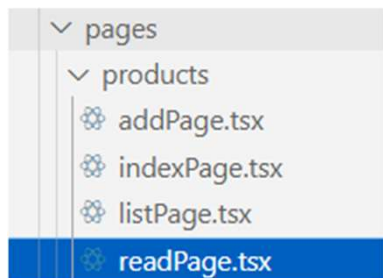
export default ListComponent;

```



조회 페이지와 조회 컴포넌트

→ loader를 이용한 상품 조회 후 컴포넌트에서 출력



```
import axios from "axios";
import { useLoaderData } from "react-router";
import { LoaderFunctionArgs } from "react-router";

export async function loadProduct({params}: LoaderFunctionArgs) {

  const {pno} = params

  const res = await axios.get(`http://localhost:8080/api/products/${pno}`)

  return res.data
}

function ReadPage() {

  const product:ProductDTO = useLoaderData()

  console.log(product)

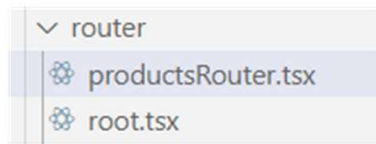
  return (
    <div className="w-full">
      <div>Product Read</div>
    </div>
  )
}

export default ReadPage
```

04 조회 페이지와 조회 컴포넌트

라우터 설정

→ router/productsRouter.tsx에 ReadPage에 대한 설정



```
import { lazy, Suspense } from "react"
import { Navigate } from "react-router"
import { loadProducts } from "../pages/products/listPage"
import { loadProduct } from "../pages/products/readPage"
...

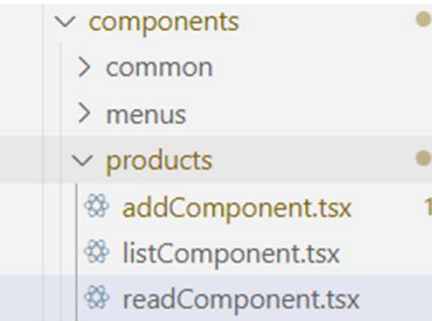
const ProductRead = lazy(() => import("../pages/products/readPage"))

const Loading = () => <div>Products Loading....</div>

export default function productsRouter() {

  return (
    {
      path: "products",
      Component: ProductsIndex,
      children: [
        {
          path: "list",
          element: <Suspense fallback={<Loading/>}><ProductsList/></Suspense>,
          loader: loadProducts
        },
        {
          path: "add",
          element: <Suspense fallback={<Loading/>}><ProductsAdd/></Suspense>
        },
        {
          path: "",
          element: <Navigate to={'/products/list'}></Navigate>
        },
        {
          path: "read/:pno",
          element: <Suspense fallback={<Loading/>}><ProductRead/></Suspense>,
          loader: loadProduct
        }
      ]
    }
  )
}
```

- 로딩된 데이터의 출력용 ReadComponent의 작성



```
import useCustomMove from "../../hooks/useCustomMove";

function ReadComponent( {product}: {product:ProductDTO}) {

  const {moveToModify, moveToList} = useCustomMove()

  return (

    <div className = "border-2 border-sky-200 mt-10 m-2 p-4 bg-white">
      <div className="flex justify-center mt-10">
        <div className="relative mb-4 flex w-full flex-wrap items-stretch">
          <div className="w-1/5 p-6 text-right font-bold">PNO</div>
          <div className="w-4/5 p-6 rounded-r border border-solid shadow-md">
            {product.pno}
          </div>
        </div>
      </div>
      <div className="flex justify-center">
        <div className="relative mb-4 flex w-full flex-wrap items-stretch">
          <div className="w-1/5 p-6 text-right font-bold">PNAME</div>
          <div className="w-4/5 p-6 rounded-r border border-solid shadow-md">
            {product.pname}
          </div>
        </div>
      </div>
    </div>

  )
}
```

```

<div className="flex justify-center">
  <div className="relative mb-4 flex w-full flex-wrap items-stretch">
    <div className="w-1/5 p-6 text-right font-bold">PRICE</div>
    <div className="w-4/5 p-6 rounded-r border border-solid shadow-md">
      {product.price}
    </div>
  </div>
</div>
<div className="flex justify-center">
  <div className="relative mb-4 flex w-full flex-wrap items-stretch">
    <div className="w-1/5 p-6 text-right font-bold">PDESC</div>
    <div className="w-4/5 p-6 rounded-r border border-solid shadow-md">
      {product.pdesc}
    </div>
  </div>
</div>
<div className="w-full justify-center flex flex-col m-auto items-center">
  {product.uploadFileNames.map( (imgFile, i) =>
    <img
      alt="product"
      key={i}
      className="p-4 w-1/2"
      src={`http://localhost:8080/api/products/view/${imgFile}`} />
  )}
</div>

<div className="flex justify-end p-4">
  <button type="button"
    className="inline-block rounded p-4 m-2 text-xl w-32 text-white bg-red-500"
    onClick={() => moveToModify(product.pno)}
  >
    Modify
  </button>
  <button type="button"
    className="rounded p-4 m-2 text-xl w-32 text-white bg-blue-500"
    onClick={() => moveToList()}
  >
    List
  </button>
</div>
</div>
)
}

```

export default ReadComponent;

Vite - React - TS

localhost:5173/products/read/23?page=1&size=10

Products

[LIST](#) [ADD](#)

Product Read

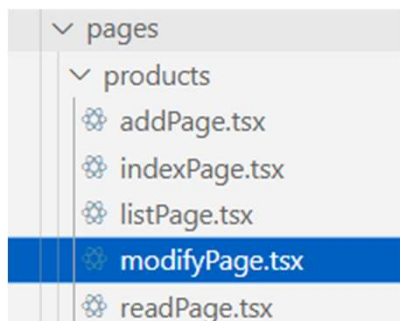
PNO	23
PNAME	test
PRICE	5600
PDESC	test




Modify List

수정/삭제를 위한 데이터 로딩은 loader를 이용해서 처리 – readPage에 있는 loader 활용

수정/삭제 작업은 useActionState()를 이용해서 처리



```
import { useLoaderData } from "react-router";

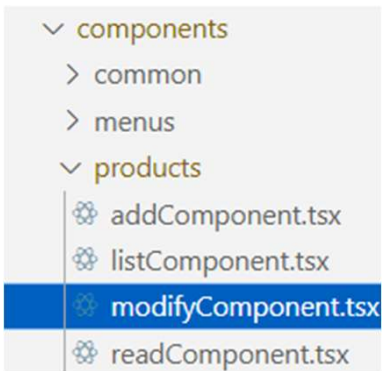
function ModifyPage() {

  const product:ProductDTO = useLoaderData()

  return (
    <div className="p-4 w-full bg-white">
      <div className="text-3xl font-extrabold">
        Products Modify Page
      </div>
    </div>
  );
}

export default ModifyPage;
```

- ModifyComponent에서는 useActionState()를 이용
- 버튼을 구분해서 동작하도록 설계



```
import { useActionState, useState } from "react";
import useCustomMove from "../../hooks/useCustomMove";
import axios from "axios";
```

```
interface ProductTaskResult {
  actionType: string,
  result: string,
  error? : string
}

const initState: ProductTaskResult = {
  actionType: 'modify',
  result: ""
}
```

```
const modifyDeleteAsyncAction = async ( state:ProductTaskResult, formData:FormData):
Promise<ProductTaskResult> => {

  const pno = formData.get("pno")

  const actionType = formData.get("actionType") as string

  let res;

  if(actionType === 'modify'){

    res = await axios.put(`http://localhost:8080/api/products/${pno}`, formData )

  }else if(actionType === 'delete') {

    res = await axios.delete(`http://localhost:8080/api/products/${pno}` )

  }

  return { actionType: actionType, result: res?.data?.RESULT}
}
```

```
function ModifyComponent( {product}: {product:ProductDTO} ) {

  const {moveToRead, moveToList} = useCustomMove()

  const [images, setImages] = useState<string[]>([...product.uploadFileNames]);

  const deleteOldImages = (event: MouseEvent<HTMLButtonElement> , target: string) => {

    event.preventDefault()
    event.stopPropagation()

    setImages(prev => prev.filter(img => img !== target));

  };

  const [state, action, isPending] = useActionState(modifyDeleteAsyncAction, initState)

  return (

    <div className = "border-2 border-sky-200 mt-10 m-2 p-4 bg-white">

      {isPending && <PendingModal/>}

      {state.result && <ResultModal title="처리완료" content="처리 완료" callbackFn={() => {

        if(state.actionType === 'modify'){
          moveToRead(product.pno)
        }
        if(state.actionType === 'delete') {
          moveToList()
        }
      }} />}

      <form action={action} >
      <div className="flex justify-center mt-10">
        <div className="relative mb-4 flex w-full flex-wrap items-stretch">
          <div className="w-1/5 p-6 text-right font-bold">PNO</div>
          <input className="w-4/5 p-6 rounded-r border border-solid border-neutral-300 shadow-md"
            name="pno" required defaultValue={product.pno}>
          </input>
        </div>
      </div>
    </form>
  )
}
```

```
<div className="flex justify-center">
  <div className="relative mb-4 flex w-full flex-wrap items-stretch">
    <div className="w-1/5 p-6 text-right font-bold">PNAME</div>
    <input className="w-4/5 p-6 rounded-r border border-solid border-neutral-300 shadow-md"
      name="pname" required defaultValue={product.pname}>
    </input>
  </div>
</div>

<div className="flex justify-center">
  <div className="relative mb-4 flex w-full flex-wrap items-stretch">
    <div className="w-1/5 p-6 text-right font-bold">PRICE</div>
    <input
      className="w-4/5 p-6 rounded-r border border-solid border-neutral-300 shadow-md"
      name="price" type={'number'} defaultValue={product.price}>
    </input>
  </div>
</div>

<div className="flex justify-center">
  <div className="relative mb-4 flex w-full flex-wrap items-stretch">
    <div className="w-1/5 p-6 text-right font-bold">PDESC</div>
    <textarea
      className="w-4/5 p-6 rounded-r border border-solid border-neutral-300 shadow-md resize-y"
      name="pdesc" rows={4} required defaultValue={product.pdesc}>
    </textarea>
  </div>
</div>
```

```

<div className="flex justify-center">
  <div className="relative mb-4 flex w-full flex-wrap items-stretch">
    <div className="w-1/5 p-6 text-right font-bold">Files</div>
    <input
      className="w-4/5 p-6 rounded-r border border-solid border-neutral-300 shadow-md"
      type='file'
      name="files"
      multiple={true}>
    </input>
  </div>
</div>

<div className="w-full justify-center flex flex-col m-auto items-center">
  {images.map( (imgFile, i) =>
    <div
      className="flex justify-center flex-col w-1/3"
      key = {i}>
      <button className="bg-blue-500 text-3xl text-white" onClick={(event) =>
deleteOldImages(event,imgFile)}>DELETE</button>
      <img
        alt ="img"
        src={`http://localhost:8080/api/products/view/s_${imgFile}`} />

      <input type="hidden" name="uploadFileNames" value={imgFile}/>

    </div>
  )}
</div>

```

```

<div className="flex justify-end p-4">
  <button type="submit" name='actionType' value='delete'
    className="rounded p-4 m-2 text-xl w-32 text-white bg-red-500">
    Delete
  </button>

  <button type="submit" name='actionType' value='modify'
    className="inline-block rounded p-4 m-2 text-xl w-32 text-white bg-orange-500">
    Modify
  </button>

  <button type="button"
    className="rounded p-4 m-2 text-xl w-32 text-white bg-blue-500"
    onClick={() => moveToList()}
  >
    List
  </button>
</div>

</form>
</div>

)
}

export default ModifyComponent;

```

Vite - React - TS

localhost:5173/products/modify/12?page=1&size=10

Main About Todo Products

Products

LIST ADD

Products Modify Page

PNO: 12

PNAME: 12번상품 수정

PRICE: 8000

PDESC: fef

Files: 파일 선택 선택된 파일 없음

DELETE

Delete Modify List

수정과 삭제에 따라 이동이
다르게 처리

처리완료

처리 완료

Close Modal

Vite - React - TS

localhost:5173/products/read/12?page=1&size=10

Main About Todo Products

Products

LIST ADD

Product Read

PNO: 12

PNAME: 12번상품 수정

PRICE: 8000

PDESC: fef



Modify List

Vite - React - TS

localhost:5173/products/list?page=1&size=10

Products

LIST ADD

Products List Page

23	 이름: test 가격: 5600	22	 이름: 테스트상품 가격: 8000
21	 이름: 테스트상품 가격: 8888	19	 이름: TEst 가격: 6000

감사합니다.